

Intent-Compiled Verified Execution for Reliable Stateful Tool Use

Anonymous ACL submission

Abstract

Language agents increasingly operate tools that mutate persistent state, but free-form tool-use loops often conflate intent interpretation, evidence acquisition, argument grounding, precondition repair, action execution, and completion detection. This creates invalid tool calls, unsafe state changes, and costly repair loops. We propose **Intent-Compiled Verified Execution (ICVE)**, a runtime approach that compiles state-sensitive intents into typed state machines while allowing uncovered low-risk requests to remain ordinary model-proposed tool calls. ICVE acquires evidence deterministically, normalizes arguments, repairs preconditions, abstains under insufficient information, executes state transitions, and stops once postconditions are satisfied. On public ToolSandbox no-distraction single-turn and insufficient-information stateful tasks, ICVE eliminates invalid tool calls and unsafe state changes while using substantially fewer model calls and tokens than ReAct. The pattern holds across weak, main, and stronger local Qwen2.5 model scales, a local non-Qwen Phi-3-mini check, and hosted DeepSeek replications. Component ablations show that the typed intent compiler and abstention verifier are both necessary for the observed gains. A small real-LLM MiniStore study shows the same direction relative to ReAct and proof-carrying tool-use, and AppWorld shows the same verified runtime transfers: local Qwen2.5-3B, hosted DeepSeek-chat, and hosted DeepSeek-reasoner intent extraction preserve 72/72 success on a targeted stateful slice, while deterministic ICVE and DeepSeek-chat intent extraction also reach 57/57 on the local official AppWorld 0.2.0 dev split with the packaged evaluator, compared with 45/57 for the official DeepSeek ReAct-code agent. On the lo-

cal AppWorld 0.2.0 `test_normal.txt` file, deterministic ICVE and DeepSeek-chat intent extraction reach 164/167 overall and 164/164 supported success, with the three failures limited to guarded abstentions.

1 Introduction

Stateful tool use differs from stateless API answering. A single wrong argument can delete the wrong contact, send a message under the wrong precondition, or repeat an already completed state transition until the execution environment raises an error. Existing tool-use agents often represent this entire process as a text loop: the language model chooses a tool, fills arguments, interprets observations, repairs errors, and decides when to stop. ReAct-style agents, tool-learning systems, and recent interactive benchmarks make this setting increasingly realistic (Yao et al., 2022; Schick et al., 2023; Lu et al., 2024; Trivedi et al., 2024; Wang et al., 2025), but they leave open where stateful execution should be verified.

Our central claim is that many failures are not reasoning failures alone. They are *execution-boundary* failures. The model is asked to perform intent compilation, state tracking, evidence lookup, precondition management, and completion detection through plain text. Increasing model scale helps but does not remove invalid calls or unsafe state changes in insufficient-information settings.

ICVE changes the execution boundary. The LLM may still propose actions for uncovered requests, but risky state-sensitive intents are handled by a runtime compiler that emits verified tool traces. The runtime owns typed slots, evidence acquisition, safe precondition repair, abstention, state transitions, and postcondition-based stopping. The unit of reuse is therefore not a prompt template but

a small executable contract: once a stateful task family has a machine, weaker or stronger models only choose a frame, while the same runtime executes and checks the state transition. This paper constructs the machines manually; the claim is an explicit, auditable, reusable boundary once registered, not solved machine synthesis.

The distinction is not merely stricter schema validation. A function schema can reject an ill-typed call, but it does not decide which evidence must be acquired before a mutation, whether an earlier transition already satisfied the goal, or whether missing state should force abstention. Nor is ICVE only semantic parsing from utterances to API calls: the compiled object is an executable state machine whose transition relation is checked against a live ledger. Finally, unlike proof-carrying prompting, the runtime does not ask the model to author the proof obligation after proposing an action. The proof burden is compiled into the machine and reused across turns and models.

This paper makes three contributions:

1. A typed intent/state-machine abstraction for stateful tool use. Each risky task family is represented as an `IntentMachine` with an explicit schema, compiler, and handler.
2. A verified runtime that owns evidence acquisition, argument grounding, precondition repair, insufficient-information abstention, and completion detection.
3. A public-benchmark evaluation, including ToolSandbox, a targeted AppWorld transfer slice, and a same-split local official AppWorld dev comparison, showing that runtime-owned execution sharply reduces invalid calls and unsafe state changes while avoiding the repair overhead that made proof-carrying tool-use unattractive in the same setting.

2 Method

2.1 Typed Intent Machines

ICVE represents each covered stateful task family as an intent machine:

```
Machine = (schema, compiler, handler)
schema = (intent_type, slots,
          terminal_states)
```

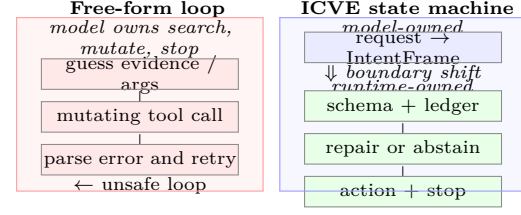


Figure 1: The execution-boundary shift. A free-form agent owns evidence search, mutation, error recovery, and stopping, so missing state can become invalid calls or unsafe loops. ICVE restricts the model to typed frame selection and moves ledger checks, repair, abstention, tool execution, and postcondition stopping into the runtime.

```
compiler = request x dialogue x tools -> frame
          or None
handler = frame x ledger x tools ->
          tool_action or final_action
```

An `IntentFrame` is the runtime-owned state for a user intent:

```
IntentFrame = (
    intent_type,
    slots: name -> (value, required, source),
    state,
    missing_slots,
    abstain_reason
)
```

The compiler is deliberately typed and conservative. It does not ask the model to emit a proof or a full tool trace. Instead, it maps a user request into a small frame whose slot values are extracted from the request, copied from prior observations, or produced by a canonicalizer. In the ToolSandbox binding, the registry currently covers 13 intent families: currency conversion, stock lookup, reverse geocode, current city, last-message contact update, bulk contact relationship update, contact name lookup, location phone lookup, holiday timestamp/countdown, weather, distance, and reminder.

The machine interface is also the method’s threat boundary. A model may choose the wrong intent type or fill a wrong slot; that risk is handled by schema validation, conservative compilers, and domain verifiers. The model is not allowed to choose arbitrary state-changing calls for a covered intent. For those intents, handlers may only emit runtime actions derived from validated slots and ledger observations, and a policy verifier can reject or rewrite the action before execution. Thus ICVE’s claim is conditional and inspectable:

correctness depends on the registered machine and verifier, not on an opaque promise that the model will reason safely.

2.2 Compilation

Compilation proceeds by registry order. Figure 1 illustrates the resulting boundary: the compiler selects a frame, while the runtime owns state checks, repair, abstention, execution, and stopping. On each turn, ICVE first runs an insufficient-information verifier. If required tools or information are unavailable, the verifier returns an abstention frame. Otherwise, the runtime tries registered compilers until one returns a frame, and validates that frame against its schema. If no registered machine applies, control falls back only for uncovered low-risk requests.

This loop is implemented in a benchmark-agnostic runtime named `RaveRuntime` in the artifact for historical reasons. Domain bindings provide intent machines plus a small runtime policy: a state-ledger interface for grounded observations and a verifier interface for final-message wording and transition rejection. The generic runtime owns the registry, uniqueness checks, frame validation, completion/abstention hooks, and handler dispatch. A policy verifier can rewrite a proposed `ToolAction` into a `FinalAction`, which gives each domain binding a single runtime boundary for rejecting unsafe or previously failed transitions.

2.3 Verified State-Machine Runtime

The runtime consumes a state ledger rather than trusting raw dialogue text. The ledger records user-provided values, successful tool observations, and recent tool errors; a `ToolSandbox` ledger instantiates this interface with settings, contacts, messages, and reminders. A state-machine handler consumes the current `IntentFrame` and ledger, then emits exactly one transition:

```
ToolAction = (tool, args, reason)
FinalAction = (message, reason)
```

For read-only actions, the verifier normalizes arguments and blocks empty or ungrounded searches. For state-changing actions, it checks schema conformance, argument grounding, state preconditions, and duplicate

completion before allowing exactly one transition. If the requested postcondition is already satisfied, the runtime stops. When a safe repair exists, such as disabling low-battery mode before enabling Wi-Fi, the runtime emits the repair action before the requested mutation. When repair would require missing information or unavailable tools, it abstains instead of letting the model guess. Thus the verifier is stricter than a call schema: it checks the transition’s evidence, preconditions, and completion state, not only the argument types of one invocation.

The design point is that stateful execution is runtime-owned. The model can still be used for uncovered or low-risk requests, but registered stateful intents are compiled and executed through deterministic state machines.

3 Experimental Setup

We evaluate under real local LLM inference rather than oracle policy traces. The main agent LLM is served through a local OpenAI-compatible CUDA server. `ToolSandbox` tools that would otherwise require external RapiAPI calls use deterministic offline fixtures so the benchmark remains repeatable.

All reported local-model experiments use temperature 0 inference on a single NVIDIA RTX 3080 Ti GPU with 12GB memory. Qwen2.5-7B and Phi-3-mini are run with 4-bit quantization; Qwen2.5-0.5B and Qwen2.5-3B are run as local instruction models through the same OpenAI-compatible interface. No model training or fine-tuning is performed.

For the hosted replication, we run the same `ToolSandbox` suites with the DeepSeek API model identifiers `deepseek-chat` and `deepseek-reasoner` through the same OpenAI-compatible client. API keys are loaded from environment variables and are not written to command lines or result files.

For AppWorld, we use two settings. First, we run an isolated `pctu-appworld` environment on a targeted public stateful slice covering 72 tasks across twenty-four registered AppWorld intent machines; some hosted DeepSeek rows are combined from an earlier 66-task run plus the final six-task Spotify-family expansion. Second, we run ICVE on the complete local official AppWorld 0.2.0 `dev.txt` split

in a separate `pctu-appworld-agents` environment and evaluate with AppWorld’s packaged evaluator. The deterministic path uses typed compilers only; the real-LLM paths use local Qwen2.5-3B, DeepSeek-chat, or DeepSeek-reasoner to produce a typed intent frame and then execute that frame through the same verified runtime. AppWorld handlers interact only through the live `apis` object; the runtime policy blocks code that references task files, ground truth, compiled solutions, or oracle data.

Metrics are task success, ToolSandbox similarity, invalid tool calls per task, unsafe state changes per task, verifier rejections, runtime repair actions, model calls, and a token proxy computed from local model usage. The kill criterion is not only safety: a candidate must reduce invalid calls and unsafe state changes without letting repair cost erase task-success or token-efficiency gains. For AppWorld, we report evaluator success, unsafe state-test failures, failed API repair attempts, API calls, model calls, and model-reported token usage.

We use Wilson 95% intervals over the fixed episode counts in each main suite; the complete interval table is archived in the supplement. Invalid and unsafe counts are reported as per-episode means on the same fixed slices.

The experiments answer three questions: whether ICVE reduces invalid or unsafe state changes, whether the same boundary transfers across models and environments, and whether typed machines plus abstention are necessary beyond intent recognition. The anonymous supplement includes source code, task-id lists, evaluator summaries, and commands for reproducing the reported rows.

Baselines. We compare against ReAct (Yao et al., 2022), a plain JSON tool-use loop with no runtime intent compiler; PCTU, a proof-carrying ablation that asks the model to attach action preconditions and lets a verifier reject unsafe steps; and ICVE ablations that remove the typed DSL or the abstention verifier.

4 Results

4.1 MiniStore Diagnostic

MiniStore is a small local stateful environment used as a diagnostic rather than as the headline benchmark. It shows why model-authored

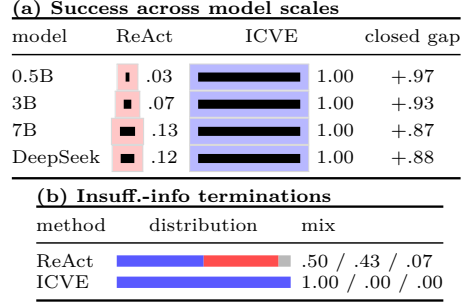


Figure 2: Deep ToolSandbox analysis. ICVE closes the success gap across weak, main, and hosted model scales and changes insufficient-information failures by eliminating unsafe state transitions. Blue denotes success, red unsafe mutation, and gray other invalid or incomplete termination.

proof obligations are not enough.

method	succ.	goal	invalid	unsafe	LLM	tokens
ReAct	0.375	1.000	1.125	1.125	5.500	2477.750
PCTU	0.250	0.250	0.000	0.000	7.250	3169.375
ICVE	1.000	1.000	0.000	0.000	2.250	603.875

Table 1: MiniStore real-LLM diagnostic with Qwen2.5-3B. PCTU suppresses invalid and unsafe actions but pays too much proof/repair overhead.

4.2 Public ToolSandbox

The strongest evidence is on ToolSandbox (Lu et al., 2024). Table 2 reports the primary Qwen2.5-3B results, and Figure 2 summarizes the scale and failure-mode effects.

The Wilson 95% confidence intervals for the primary success rates are separated from the mean-count columns: on the 73-episode single-turn suite, ICVE is 1.0000 [0.9500, 1.0000] while ReAct is 0.0685 [0.0296, 0.1505]; on the 28-episode insufficient-information suite, ICVE is 1.0000 [0.8794, 1.0000] while ReAct and the no-abstention ablation are both 0.5000 [0.3263, 0.6737].

4.3 Model Scale

The effect is not a Qwen2.5-3B-only artifact. Table 3 summarizes weak and stronger local model runs. The 7B result uses a local 4-bit quantized model and should not be interpreted as frontier-model coverage.

4.4 Cross-Family Diagnostic

We also ran a small non-Qwen diagnostic with Phi-3-mini. Table 4 reports 10 core ToolSandbox scenarios and 10 insufficient-information scenarios. This is not frontier-model evidence,

suite	method	n	succ.	invalid	unsafe	repair	LLM	tokens
single-turn	ReAct	73	0.0685	2.1507	0.0000	0.0000	11.8219	11048.0
single-turn	ICVE no DSL	73	0.4932	0.3288	0.0000	0.8082	3.9726	3307.9
single-turn	ICVE	73	1.0000	0.0000	0.0000	2.4658	0.1233	32.7
insufficient-info	ReAct	28	0.5000	2.1786	0.4286	0.0000	11.7500	9896.4
insufficient-info	ICVE no abstention	28	0.5000	1.5714	0.4286	0.3929	7.2857	6612.1
insufficient-info	ICVE	28	1.0000	0.0000	0.0000	1.5714	0.0000	0.0

Table 2: Main public ToolSandbox results with Qwen2.5-3B. Metrics after success are per-episode means. ICVE performs deterministic repair actions but still uses far fewer model calls and tokens; the typed DSL is necessary on single-turn tasks, and the abstention verifier is necessary on insufficient-information tasks.

model	suite	method	n	succ.	invalid	unsafe	tokens
0.5B	single-turn	ReAct	30	0.0333	5.6667	0.0000	8054.5
0.5B	single-turn	ICVE	30	1.0000	0.0000	0.0000	54.3
0.5B	insufficient-info	ReAct	28	0.6071	3.2500	0.3214	8367.3
0.5B	insufficient-info	ICVE	28	1.0000	0.0000	0.0000	0.0
7B-4bit	single-turn	ReAct	30	0.1333	0.4333	0.0000	3483.5
7B-4bit	single-turn	ICVE	30	1.0000	0.0000	0.0000	34.6
7B-4bit	insufficient-info	ReAct	28	0.4286	1.2143	0.3929	4894.4
7B-4bit	insufficient-info	ICVE	28	1.0000	0.0000	0.0000	0.0

Table 3: ICVE preserves the zero-invalid, zero-unsafe pattern across weak and stronger local Qwen2.5 scales; the primary Qwen2.5-3B results appear in Table 2.

but it addresses the narrow concern that the result is only a Qwen-family artifact.

4.5 Hosted OpenAI-Compatible Replication

We also replicate the primary ToolSandbox suites with the hosted DeepSeek-chat endpoint. Table 5 shows the same qualitative pattern: ICVE reaches 1.0000 success with zero invalid calls and zero unsafe state changes on both suites, while ReAct remains substantially less successful and more expensive. The no-DSL and no-abstention ablations again expose the two mechanisms: typed compilation is needed on single-turn stateful tasks, and abstention verification is needed when the user omits required information.

We also ran DeepSeek-reasoner on the same two suites; both full runs passed the kill criteria, and the recorded summaries are listed in the artifact manifest.

4.6 AppWorld

The same verified runtime transfers to AppWorld on a targeted public state-changing slice. The active rows cover 72 tasks across twenty-four registered AppWorld intent machines spanning phone, Venmo, Gmail, Simple Note, Spotify, Amazon, alarms, Bucket List, and cross-app sharing tasks.

The AppWorld result separates intent recognition from verified execution. Typed-intent

code gives the model the same frame but still asks it to write the mutation code; it reaches 0/72 with Qwen2.5-3B and 7/72 with DeepSeek-chat. Direct code and multi-step code-observation variants likewise fail or become expensive, while DeepSeek repair reaches 21/72 but introduces unsafe changes. The error pattern is consistent: models often know the requested operation but fail at grounded execution. One unverified DeepSeek-chat intent run extracted `archive` for a delete instruction; the instruction-aware slot verifier rewrites the frame before execution. ICVE’s residual repair cost is concentrated in Venmo transfers, where public metadata reveals card expiry but not hidden balances, so the runtime probes non-expired cards and then stops after success.

We then moved the ICVE binding to the complete local official AppWorld 0.2.0 `dev.txt` split. This adds the remaining official dev task families as registered intent machines, including navigation-based Spotify artist lookup, Venmo social-feed aggregation, file-prefix moves, current-artist follower lookup, and Simple Note markdown export. Table 7 reports AppWorld’s packaged evaluator on all 57 dev tasks. Both deterministic ICVE and ICVE with DeepSeek-chat intent extraction reach 57/57 local success and 100.0 task-goal / 100.0 scenario-goal comple-

suite	method	n	succ.	invalid	unsafe	tokens
core10	ReAct	10	0.0000	1.8000	0.0000	6072.0
core10	ICVE	10	1.0000	0.0000	0.0000	87.9
insufficient10	ReAct	10	0.6000	1.7000	0.3000	5216.3
insufficient10	ICVE no abstention	10	0.6000	0.9000	0.4000	1123.2
insufficient10	ICVE	10	1.0000	0.0000	0.0000	0.0

Table 4: Phi-3-mini cross-family diagnostic. The typed runtime remains clean on both slices, and removing the abstention verifier again permits invalid and unsafe attempts.

suite	method	n	succ.	invalid	unsafe	tokens
single-turn	ReAct	73	0.1233	0.5342	0.0000	4397.1
single-turn	ICVE no DSL	73	0.5479	0.0548	0.0000	1258.6
single-turn	ICVE	73	1.0000	0.0000	0.0000	13.6
insufficient-info	ReAct	28	0.3214	1.2143	0.6071	7299.6
insufficient-info	ICVE no abstention	28	0.5000	0.6786	0.4286	3234.4
insufficient-info	ICVE	28	1.0000	0.0000	0.0000	0.0

Table 5: Hosted DeepSeek-chat ToolSandbox replication. Metrics after success are per-episode means; full outputs are archived in the artifact manifest.

tion, while the official DeepSeek ReAct-code agent reaches 79.0 / 73.7 and 45/57. The residual 0.0351 invalid API attempts per task are two Venmo payment-card probes: public card metadata exposes expiry but not balance, so the runtime tries one non-expired insufficient-balance card and then succeeds with the next public card. We do not use the private admin balance API to remove this residual repair cost.

The official DeepSeek ReAct-code dev57 run completes all tasks in 75.9 minutes, with 879 LLM calls and 6.45M tokens. For context, its earlier dev10_full150 slice reached 90.0 / 75.0 with 174 model calls and 1.40M tokens, while ICVE rows reached 100.0/100.0. These are same-split local official-dev comparisons, not held-out test or leaderboard submissions.

As a held-out smoke, we then expanded beyond the official dev split to the beginning of AppWorld `test_normal`. The first-12 smoke initially exposed zero coverage; after registering four typed machines, deterministic ICVE reached 12/12 with zero invalid tool calls and zero unsafe state changes. We further expanded the smoke to the first 120 `test_normal` tasks. After registering additional Simple Note and Venmo machines plus Simple Note-to-Spotify playlist, release-month, reunion-RSVP file, phone-sourced Spotify playlist update, roommate dinner-settlement, offline playback, and Venmo card-path repair machines, the registry supports 117/120 and succeeds on all 117/117 supported tasks, giving 117/120 overall success, 0

unsafe state changes, and 0.0250 invalid/tool per task. The invalid count consists of 3 unsupported or no-action abstentions from the unsupported vacation-settlement family. The Venmo named-payment family now adds an available public supervisor card before transfer instead of probing failing cards, and the roommate payment-request family rotates successful non-expired cards to avoid reusing a depleted card first. We report this as held-out-prefix smoke evidence rather than a leaderboard result. DeepSeek-chat intent extraction on the same first 120 tasks matches this deterministic coverage profile after the instruction-aware abstention verifier is applied: 117/120 overall success, 117/120 supported, 117/117 supported success, 0 unsafe state changes, 0.0250 invalid/tool per task, and 1 LLM call per task. A strict verifier blocks unsupported vacation-settlement tasks before code execution and counts them as invalid no-action results. Earlier LLM over-generalizations on unsupported housing-bill, offline-playback, and carpool Venmo tasks are now either converted to abstention or replaced by dedicated typed machines rather than similar-looking state changes. Running the remaining local `test_normal.txt` entries gives the same profile at file level: both deterministic ICVE and DeepSeek-chat intent extraction reach 164/167 overall, 164/167 supported, and 164/164 supported success with 0 unsafe state changes; the only failures are the same three protected vacation-settlement abstentions. The 121–167 continuation covers the remaining 47 entries

method	n	succ.	inv./repair	unsafe	LLM	tokens
deterministic compiler	72	72/72	0.0694	0.0000	0.0000	180.5
Qwen2.5-3B intent	72	72/72	0.0694	0.0000	1.0000	3220.3
DeepSeek-chat intent	72	72/72	0.0694	0.0000	1.0000	3352.2
DeepSeek-reasoner intent	72	72/72	0.0694	0.0000	1.0000	3462.3
Qwen direct code	72	0/72	1.0000	0.0000	1.0000	1544.0
Qwen typed intent/code	72	0/72	1.0000	0.0000	2.0000	4674.8
Qwen code repair	72	0/72	2.7083	0.0000	2.8889	5342.8
Qwen multi-step code	72	0/72	3.7500	0.0000	4.3472	9031.7
DeepSeek direct code	72	4/72	0.8611	0.0139	1.0000	2028.9
DeepSeek typed intent/code	72	7/72	0.7917	0.0139	2.0000	5478.9
DeepSeek code repair	72	21/72	1.5556	0.0417	2.3472	5898.8
DeepSeek multi-step code	72	20/72	1.2222	0.0000	4.3056	12128.6

Table 6: Targeted public AppWorld slice. Metrics after success are per-task means. All rows use the same 72-task dataset; hosted DeepSeek rows are combined from incrementally flushed shards where needed. AppWorld’s packaged evaluator confirms the corresponding task-completion counts: 72/72 for ICVE runtime rows, 0/72 for local Qwen code baselines, 4/72 for DeepSeek direct code, 7/72 for DeepSeek typed intent/code, 21/72 for DeepSeek repair, and 20/72 for DeepSeek multi-step code.

method	n	task	scen.	invalid	unsafe	LLM
deterministic compiler	57	100.0	100.0	0.0351	0.0000	0.0000
DeepSeek-chat intent	57	100.0	100.0	0.0351	0.0000	1.0000
official DeepSeek ReAct-code	57	79.0	73.7	–	–	15.4211

Table 7: Complete local official AppWorld 0.2.0 dev split for ICVE rows. Task and scenario columns are packaged-evaluator percentages; invalid, unsafe, and LLM are per-task means from the local runtime logs for ICVE and official runner logs for ReAct-code. The official ReAct-code row uses 6.45M tokens total.

in the local file and reaches 47/47 overall success and 47/47 supported success, with 0 unsafe state changes and 0 invalid tool calls. A full DeepSeek-chat intent-extraction companion run on the same 47 tasks matches this result with 1 LLM call per task, 0 unsafe state changes, and 0 invalid tool calls. This file-level continuation remains a coverage diagnostic rather than a full held-out test split or leaderboard result. We also ran a separate beginning-of-test_challenge Amazon diagnostic after adding two conservative saved-list machines: the first 24 tasks improved from 3/24 to 12/24 success with zero unsafe state changes, while product-search purchase requests that require choosing among sellers, dimensions, or multiple candidate products remain unsupported.

4.7 Multi-Turn Diagnostic

On a 28-episode hidden-task multi-turn diagnostic with Qwen2.5-0.5B, ICVE reaches 0.8929 success, 0.9773 mean similarity, 0 invalid calls, and 0 unsafe state changes. This is an extension result, not the headline claim: a small number of recency and benchmark-interface near-misses remain.

5 Analysis

The empirical pattern is that free-form agents overpay for tasks with simple state semantics. ReAct must repeatedly infer which evidence is needed, call tools with fragile arguments, interpret observations, decide whether preconditions are satisfied, and detect completion. PCTU improves safety by rejecting bad calls, but it still relies on the model to propose proof-bearing actions. ICVE moves these repeated proof obligations into the runtime. Runtime actions increase, but model calls and token usage collapse because the agent avoids repeated model-mediated repair.

6 Related Work

ReAct-style agents interleave reasoning and action through text (Yao et al., 2022). Reflexion adds verbal self-feedback (Shinn et al., 2023), and Toolformer explores learning tool use from self-supervised signals (Schick et al., 2023). Recent tool-learning systems and datasets further scale API selection and function calling (Li et al., 2023; Shen et al., 2024; Qin et al., 2024; Patil et al., 2024; Liu et al., 2025; Dang et al., 2025). These lines improve model behavior, but they still leave stateful execution largely model-owned. ICVE instead changes

the execution boundary for covered risky intents.

Tool-use benchmarks have become more realistic. ToolSandbox introduces stateful, interactive evaluation with insufficient-information and canonicalization cases (Lu et al., 2024). AppWorld, BFCL, and tau-bench broaden interactive and function-calling evaluation (Trivedi et al., 2024; BFCL; tau-bench); broader agent benchmarks stress multi-step interaction in text, web, and operating-system environments (Liu et al., 2024; Zhou et al., 2024; Xie et al., 2024). Recent stateful-dialogue tool-use benchmarks further show that state tracking remains difficult (Wang et al., 2025). ICVE is complementary: it is a runtime method intended to reduce invalid and unsafe execution in such environments.

Guardrails, function schemas, and sandboxed risk evaluation validate or probe tool-agent behavior (Ruan et al., 2024; Xie et al., 2025; Faghih et al., 2025). ICVE verifies traces against intent-level state machines: the runtime checks not only whether one call is syntactically valid, but whether it is grounded in the requested state change and whether the postcondition has already been satisfied. This shifts the unit of verification from one tool invocation to a stateful intent. The AppWorld typed-intent-code ablation highlights the distinction: giving the model a valid frame but letting it write the mutation code still fails on most tasks, because the difficult part is not only recognizing the intent but executing the grounded state transition.

Semantic parsing and task-oriented dialogue systems also use intent and slot structure, but their usual target is interpreting user utterances or filling API arguments. ICVE uses typed intent frames as an execution substrate: compilers produce state-machine frames, handlers acquire missing evidence and repair preconditions, and the verifier can abstain when the state needed for safe execution is unavailable. This is why the ablation without the typed DSL loses both success and invalid-call control in Table 2.

Proof-carrying tool-use approaches ask the model to make safety obligations explicit. Our MiniStore diagnostic shows the limitation of that boundary: proof checks can suppress invalid and unsafe mutations, but the model still

pays for repeated proof generation, rejection, and repair. ICVE keeps the verification boundary while moving recurring proof obligations into deterministic machines.

7 Conclusion

ICVE reframes reliable stateful tool use as verified execution, not as longer prompting or model-authored proofs. The agent supplies intent; a typed runtime owns risky state transitions. On the covered public ToolSandbox slices, this boundary reduces invalid calls and unsafe state changes while improving task success and reducing repair cost. AppWorld transfer, official-dev, and local file-level runs show the same execution boundary beyond ToolSandbox. Public leaderboard evaluation and broader real-LLM coverage remain the next empirical step.

Limitations

The current strongest claim is intentionally narrow. The clean real-LLM evidence is on ToolSandbox no-distraction single-turn and insufficient-information suites plus targeted AppWorld transfer, complete local official AppWorld dev57, and local `test_normal.txt` file-level runs. These runs use AppWorld’s packaged evaluator where applicable, but they are not a public AppWorld leaderboard submission. Unsupported vacation-settlement tasks remain blocked as invalid no-action results rather than being forced through a similar-looking Venmo machine. Multi-turn coverage is promising but not yet broad enough for a headline claim. The current IntentMachine registry is explicit and partly hand-engineered; future work should learn or synthesize candidate machines and then verify them.

The runtime does not eliminate the need for a language model. It changes where the model is trusted. ICVE still needs the model for uncovered intents, natural-language clarification, and domains whose state machines have not been registered. The safety claim therefore applies to covered high-risk intents, not arbitrary tool execution.

Finally, hosted-model coverage remains narrow: the evidence spans Qwen2.5 scales, a small Phi-3-mini diagnostic, and hosted

DeepSeek replications, but not a broad set of proprietary frontier models.

Ethical Considerations

Reliable stateful tool execution has positive applications in reducing accidental mutations, repeated tool calls, and unsafe guessing by deployed agents. The same runtime discipline could also make high-impact automated systems more capable, so the method should be paired with domain authorization, audit logs, and tool-level access controls. ICVE is not a substitute for policy enforcement: it verifies covered state transitions against declared machine-readable rules, but cannot guarantee safety for tools or intents outside the registry.

Reproducibility Statement

All reported benchmark results are produced by local scripts and archived output directories listed in the artifact manifest. For AppWorld, the implementation uses public task instructions, public API documentation, and live public app state observed through the benchmark APIs. It does not read hidden answers, private evaluation code, oracle solutions, or compiled solutions when constructing actions. Unsupported or ambiguous high-risk families are reported as abstentions rather than forced executions. The anonymous supplement includes source code, task-id lists, evaluator summaries, and environment notes, but excludes API keys and hosted-model credentials. It also avoids redistributing AppWorld’s protected bundle, raw databases, access tokens, raw task logs, or downloaded data bundles; users should obtain AppWorld through its official release channel and follow its license and redistribution terms.

References

BFCL. 2024. Berkeley function calling leaderboard. [Berkeley Function Calling Leaderboard](#).

Dang et al. 2025. Improving large language models function calling and interpretability via guided-structured templates. In *EMNLP*.

Faghih et al. 2025. Tool preferences in agentic LLMs are unreliable. In *EMNLP*.

Li et al. 2023. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *EMNLP*.

Liu et al. 2024. AgentBench: Evaluating LLMs as agents. In *ICLR*.

Liu et al. 2025. ToolACE: Winning the points of LLM function calling. In *ICLR*.

Lu et al. 2024. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. *arXiv preprint arXiv:2408.04682*.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. In *NeurIPS*.

Qin et al. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *ICLR*.

Ruan et al. 2024. Identifying the risks of LM agents with an LM-emulated sandbox. In *ICLR*.

Schick et al. 2023. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.

Shen et al. 2024. Small LLMs are weak tool learners: A multi-LLM agent. In *EMNLP*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*.

tau-bench. 2024. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.

Trivedi et al. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *ACL*.

Wang et al. 2025. Rethinking stateful tool use in multi-turn dialogues: Benchmarks and challenges. *arXiv preprint arXiv:2505.13328*.

Xie et al. 2024. OSWorld: Benchmarking multi-modal agents for open-ended tasks in real computer environments. In *NeurIPS*.

Xie et al. 2025. ToolSafety: A comprehensive dataset for enhancing safety in LLM-based agent tool invocations. In *EMNLP*.

Yao et al. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Zhou et al. 2024. WebArena: A realistic web environment for building autonomous agents. In *ICLR*.